

Домашняя работа 04 Гареевой Фариды

ClickHouse Homework. UDF, Aggregate Functions and working with data types

Вариант 1:

Цель:

Цель этого домашнего задания - помочь вам понять и применить агрегатные функции, функции, работающие с типами данных, и функции, определяемые пользователем (UDF) в ClickHouse.

Задачи:

Вам предстоит выполнить следующие задания:

1. Используйте агрегатные функции для обобщения данных.
2. Применять функции, работающие с различными типами данных.
3. Создавать и использовать функции, определяемые пользователем (UDF), в ClickHouse.

Набор данных:

Для выполнения этого домашнего задания вы будете использовать пример набора данных, представляющего транзакции электронной коммерции. Предположим, что у вас есть таблица `transactions` со следующей схемой:

```
CREATE TABLE transactions (  
    transaction_id UInt32,  
    user_id UInt32,  
    product_id UInt32,  
    quantity UInt8,  
    price Float32,  
    transaction_date Date  
) ENGINE = MergeTree()  
ORDER BY (transaction_id);
```

Задание:

1. Агрегатные функции

1. Рассчитайте общий доход от всех операций.
2. Найдите средний доход с одной сделки.
3. Определите общее количество проданной продукции.
4. Подсчитайте количество уникальных пользователей, совершивших покупку.

2. Функции для работы с типами данных

1. Преобразуйте `transaction\_date` в строку формата `YYYY-MM-DD`.
2. Извлеките год и месяц из `transaction\_date`.
3. Округлите `price` до ближайшего целого числа.
4. Преобразуйте `transaction\_id` в строку.

3. User-Defined Functions (UDFs)

1. Создайте простую UDF для расчета общей стоимости транзакции.
2. Используйте созданную UDF для расчета общей цены для каждой транзакции.
3. Создайте UDF для классификации транзакций на «высокоценные» и «малоценные» на основе порогового значения (например, 100).
4. Примените UDF для категоризации каждой транзакции.

Результат работы:

Выполнила запрос на создание таблицы, заполнила случайными данными на 100 млн следующим запросом:

```
insert into
hw04.transactions(transaction_id,user_id,product_id,quantity,price,transaction_date)
select number as transaction_id,
rand() % 10000 as user_id,
rand() % 5000 as product_id,
10+(rand() % 300) as quantity,
(10000+(rand() % 100000))/100 as price,
toDate(toInt32(today())-(rand() % 365)) as transaction_date
from numbers(1,100000000);
```

1. Агрегатные функции

1. Рассчитайте общий доход от всех операций.

```
select sum(quantity*price) from hw04.transactions;
```

The screenshot shows a SQL query editor with the query `select sum(quantity*price) from hw04.transactions;` and its result in a table. The table has one row with the value 6 804 709 558 876,535.

Таблица	123 sum(multiply(quantity, price))
1	6 804 709 558 876,535

2. Найдите средний доход с одной сделки.

```
select sum(quantity*price)/count() from hw04.transactions;
```

The screenshot shows a SQL query editor with the query `select sum(quantity*price)/count() from hw04.transactions;` and its result in a table. The table has one row with the value 68 047,0955887654.

Таблица	123 divide(sum(multiply(quantity, price)), count())
1	68 047,0955887654

3. Определите общее количество проданной продукции.

```
select sum(quantity) from hw04.transactions;
```

The screenshot shows a SQL query editor with the query `select sum(quantity) from hw04.transactions;` and its result in a table. The table has one row with the value 11 341 456 962.

Таблица	123 sum(quantity)
1	11 341 456 962

4. Подсчитайте количество уникальных пользователей, совершивших покупку.

```
select uniq(user_id) from hw04.transactions;
select uniqExact(user_id) from hw04.transactions;
```

The screenshot shows a SQL query editor with the query `select uniq(user_id) from hw04.transactions;` and its result in a table. The table has one row with the value 10 000.

Таблица	123 uniq(user_id)
1	10 000

2. Функции для работы с типами данных

1. Преобразуйте `transaction\_date` в строку формата `YYYY-MM-DD`.

Поскольку тип колонки Date, то достаточно просто toString()

```
select toString(transaction_date) from hw04.transactions;
```

Если бы тип колонки был DateTime, то можно вот так

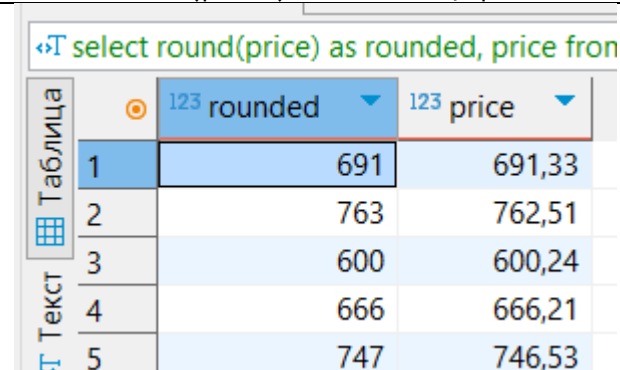
```
select formatDateTime(transaction_date, '%Y-%m-%d') from hw04.transactions;
```

2. Извлеките год и месяц из `transaction\_date`.

```
SELECT  
    toYear(transaction_date) AS year,  
    toMonth(transaction_date) AS month  
from hw04.transactions;
```

3. Округлите `price` до ближайшего целого числа.

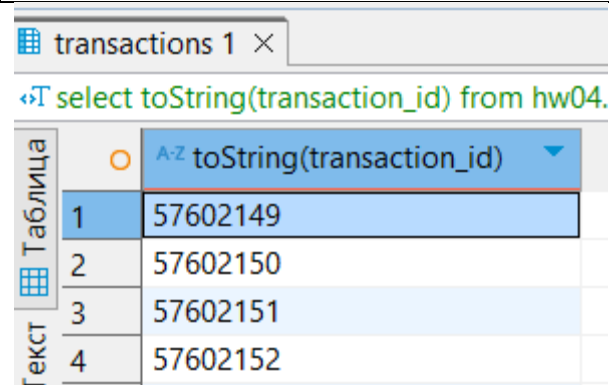
```
select round(price) as rounded, price from hw04.transactions;
```



	rounded	price
1	691	691,33
2	763	762,51
3	600	600,24
4	666	666,21
5	747	746,53

4. Преобразуйте `transaction\_id` в строку.

```
select toString(transaction_id) from hw04.transactions;
```



	toString(transaction_id)
1	57602149
2	57602150
3	57602151
4	57602152

3. User-Defined Functions (UDFs)

1. Создайте простую UDF для расчета общей стоимости транзакции.

```
CREATE FUNCTION calc_cost AS (price, quantity) -> price*quantity;
```

```
CREATE FUNCTION calc_cost AS (price, quantity) -> price*quantity;
select *
from system.functions
where origin = 'SQLUserDefined';
```

functions 1 ×

select * from system.functions where origin: Введите SQL выражение чтобы отфильтровать результаты

A-Z name	123 is_aggregate	123 case_insensitive	A-Z alias_to	A-Z create_query
calc_cost	0	0		CREATE FUNCTION calc_cost AS (pri

2.Используйте созданную UDF для расчета общей цены для каждой транзакции.

```
select calc_cost(price, quantity), price, quantity from hw04.transactions;
```

transactions 1 ×

select calc_cost(price, quantity), price, quant Введите SQL выражение чтобы отфильтровать результ

123 calc_cost(price, quantity)	123 price	123 quantity
167 993,194152832	691,33	243
46 513,1105957031	762,51	61
20 408,1596679688	600,24	34
87 273,512878418	666,21	131
47 031,3918457031	746,53	63

3.Создайте UDF для классификации транзакций на «высокоценные» и «малоценные» на основе порогового значения (например, 100).

```
create function get_transaction_category AS (cost,cost_limit) ->
if(cost>=cost_limit,'высокоценная','малоценная');
```

```
select *
from system.functions
where origin = 'SQLUserDefined';
```

functions 1 ×

select * from system.functions where origin: Введите SQL выражение чтобы отфильтровать результ

A-Z name	123 is_aggregate	123 case_insensitive	A-Z alias_to	A-Z create_query
get_transaction_category	0	0		CREATE FUNC
calc_cost	0	0		CREATE FUNC

4.Примените UDF для категоризации каждой транзакции.

```
select calc_cost(price, quantity) as cost,
       get_transaction_category (cost, 100000) as category
from hw04.transactions;
```

transactions 1

```
select calc_cost(price, quantity) as cost, get_category as category from transactions
```

	cost	category
1	20 567,7885131836	высокоценная
2	21 021,1201171875	малоценная
3	59 746,7095336914	малоценная
4	219 059,033203125	высокоценная
5	3 690,4000854492	малоценная
6	106 704,753112793	высокоценная